HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

OOP LAB REPORT
AIMS SYSTEM

Student: Do Thanh Trung
Student ID: 202416833
Class: 757054
Instructor: Ho Viet Duc Luong

Hanoi, December 21, 2025

Contents

1 Use Case Diagram of AIMS System

1.1 System Overview and Objectives

The AIMS (An Internet Media Store) is a software application designed to manage the trading of media assets including Books, Compact Discs (CDs), and Digital Video Discs (DVDs). The system's primary goal is to simulate a store workflow where users can browse inventory, manage a shopping cart, place orders, and manage the store's stock.

1.2 Actors and Use Cases

Based on the provided Use Case Diagram, the system's functionalities are grouped by the actors who perform them:

- **Customer (User):** The primary actor who shops in the store.
  - **Browse/Search Medias:** View available items and search by title.
  - **View Details:** See specific information about a media item.
  - **Play Media:** Preview content (only for CDs and DVDs).
  - **Manage Cart:** Add items to the cart, remove items, and view total cost.
  - **Place Orders:** Finalize the purchase and clear the cart.
- **Store Manager (Admin):** The actor responsible for inventory and order management.
  - **Manage Media Store:** Add new books, CDs, or DVDs to the store, and remove existing items.
  - **Manage Orders:** Review and process orders placed by customers.
  - **Add Media Information:** Update details for specific media items.
- **Card Payment System:** An external actor that participates in the "Pay via Credit Card" use case to authorize transactions.

1.3 Diagram

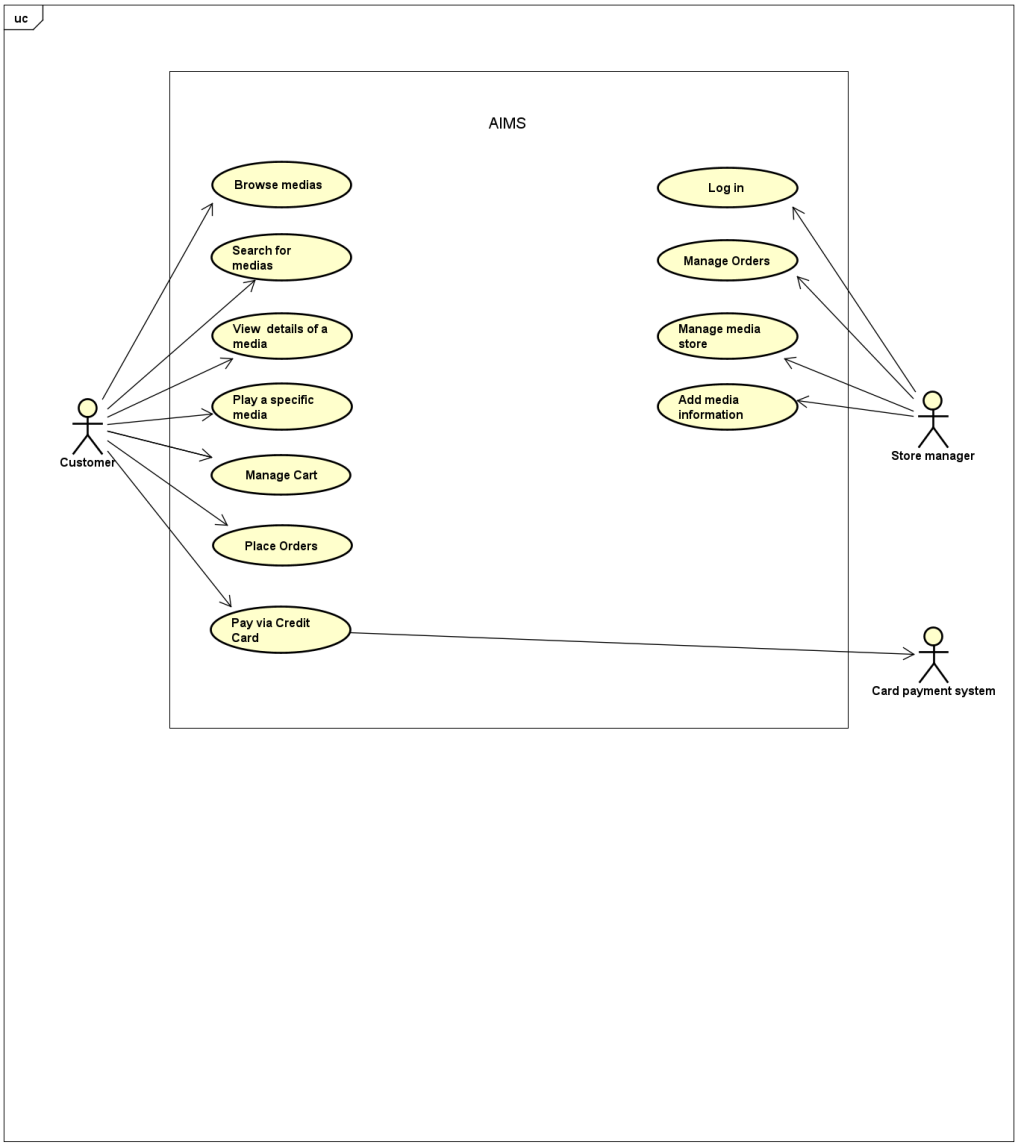

Figure 1: Use Case Diagram of AIMS System

1.4 Diagram Analysis

The diagram correctly identifies the separation of concerns between the **Customer** (consumption features) and **Store Manager** (management features). The connection between "Place Orders" and "Customer" accurately reflects the logic in `CartScreenController.java`. The "Play" use case is correctly modeled as a specific action available for certain media types.

2 Class Diagram of AIMS System

2.1 Purpose

To define the static structure of the system, illustrating the inheritance hierarchy of Media assets, the implementation of interfaces, and the relationships between the main controller classes and data models.

2.2 Diagram

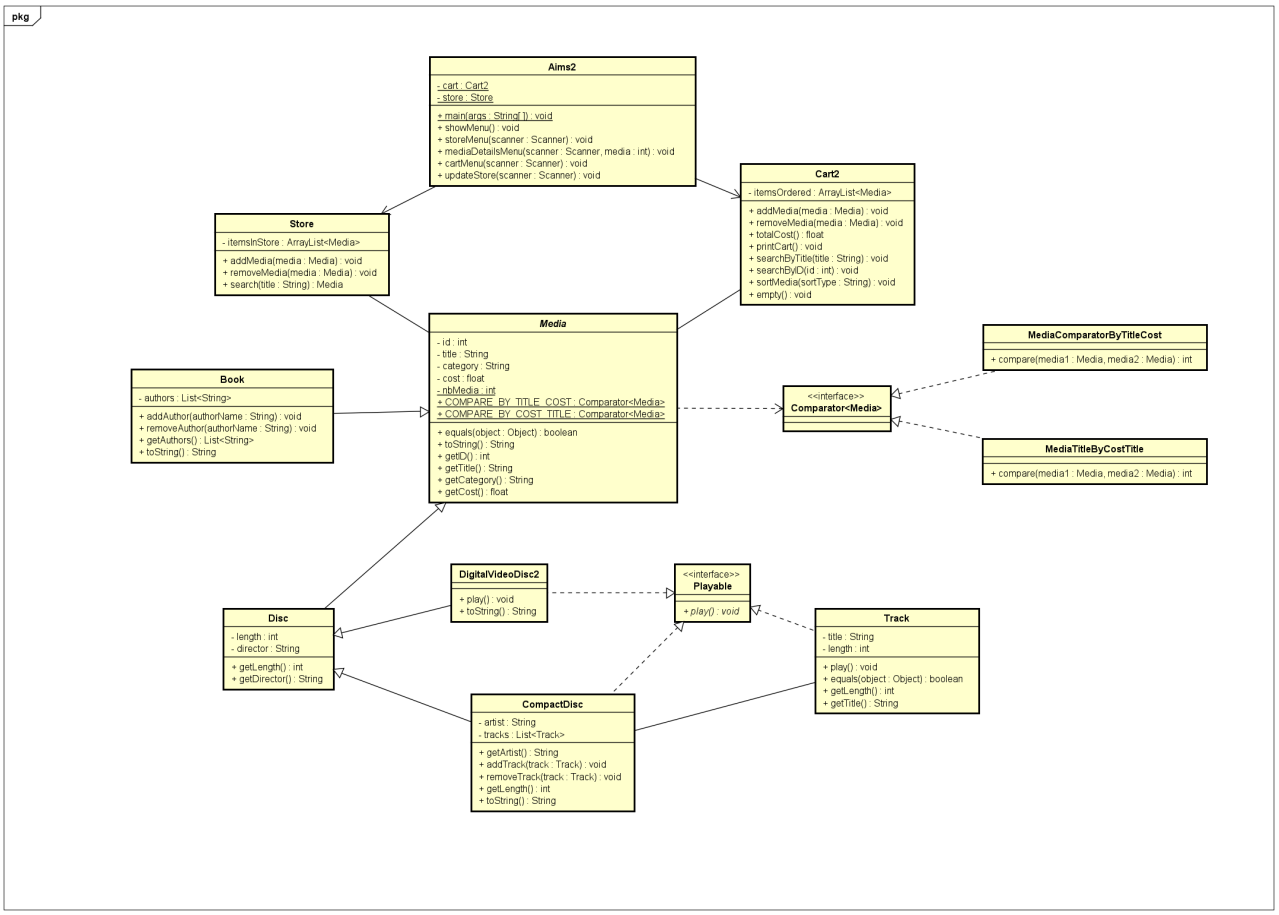

Figure 2: Class Diagram of AIMS System

2.3 Analysis of Class Structure

Based on the provided Class Diagram, the system is structured around the following key Object-Oriented principles:

2.3.1 Inheritance Hierarchy

The `Media` class serves as the abstract root of the hierarchy, defining common attributes such as `id`, `title`, `category`, and `cost`.

- `Book`**:** Extends `Media` and adds an `authors` list.

- `Disc`: Extends `Media` to include `length` and `director`, serving as a base for disc-based media.
- `DigitalVideoDisc2` & `CompactDisc`: Both extend `Disc`, inheriting its properties while adding specific behaviors.

2.3.2 Interface Implementation (Polymorphism)

The `Playable` interface defines the `play()` method. This decouples the "play" functionality from the general `Media` class.

- `DigitalVideoDisc2` and `CompactDisc` implement `Playable`, allowing users to preview these items.
- `Book` does **not** implement `Playable`, ensuring logical consistency (books cannot be played).
- `Track` also implements `Playable`, allowing individual songs on a CD to be played.

2.3.3 Sorting Strategy

The diagram illustrates the use of the `Comparator` interface to implement flexible sorting strategies. The `Media` class relies on two specific comparator implementations:

- `MediaComparatorByTitleCost`: Sorts primarily by title, then by cost.
- `MediaTitleByCostTitle`: Sorts primarily by cost, then by title.

2.4 Description of Key Classes

- `Aims2`: The main controller class containing the application entry point (`main`). It manages the navigation flow through methods like `showMenu()`, `storeMenu()`, and `cartMenu()`.
- `Store`: Maintains the inventory of the shop via the `itemsInStore` list. It provides methods to `addMedia()` and `removeMedia()`.
- `Cart2`: Manages the user's shopping session. It includes functionality to search for items (`searchByID`, `searchByTitle`), sort items (`sortMedia`), and calculate the `totalCost`.
- `CompactDisc`: Represents a music album. It maintains a composition relationship with the `Track` class (one CD contains many Tracks).

3 Application Implementation

3.1 System Initialization

Upon startup, the `Aims2.main()` method initializes the `Store` object and pre-loads it with sample data (DVDs, Books, and CDs). The application then launches the user interface (Console Menu or JavaFX GUI), allowing immediate interaction with the inventory.

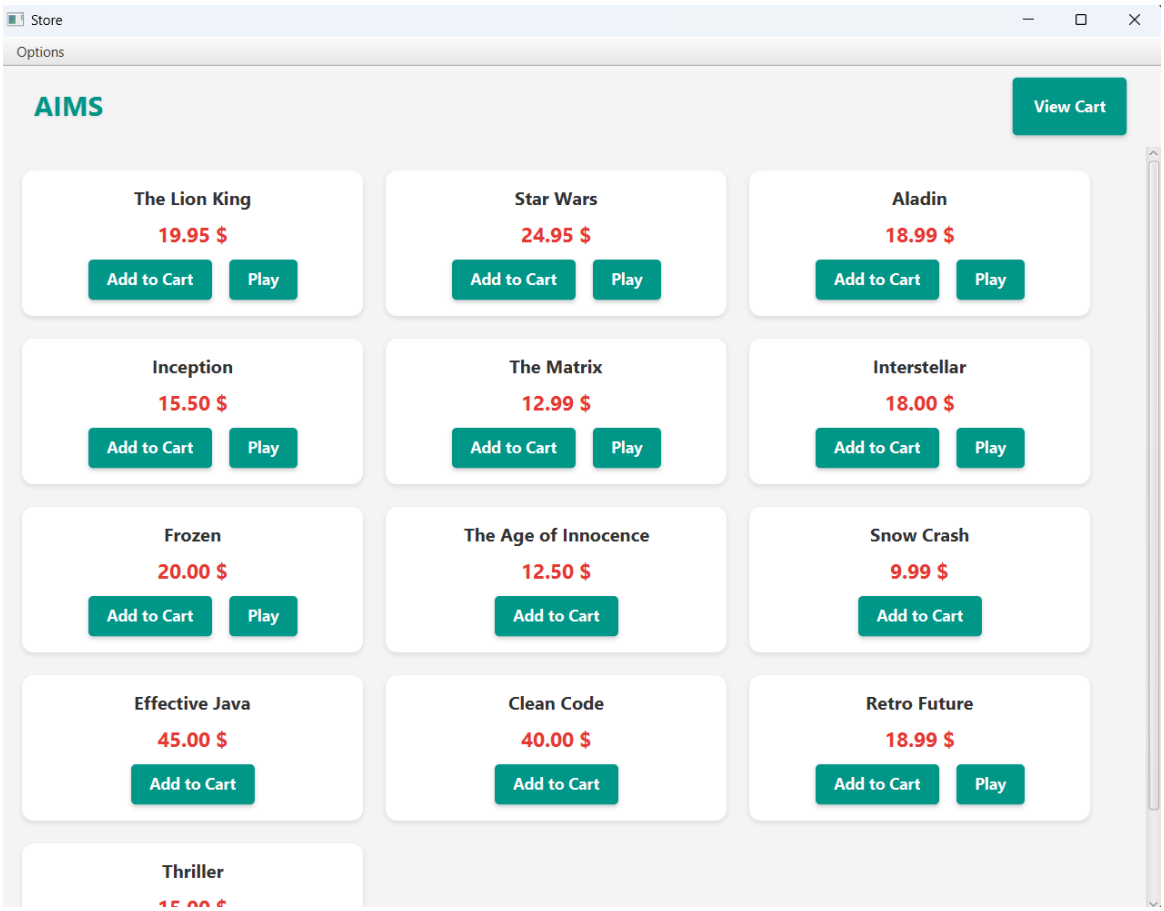

Figure 3: Store Initialization Screen

- Show the title and cost of media in the store. There is an **Add to Cart** and a **Play** button for each item. For media like Books, there is not a **Play** button.

3.2 "Add to Cart" Functionality

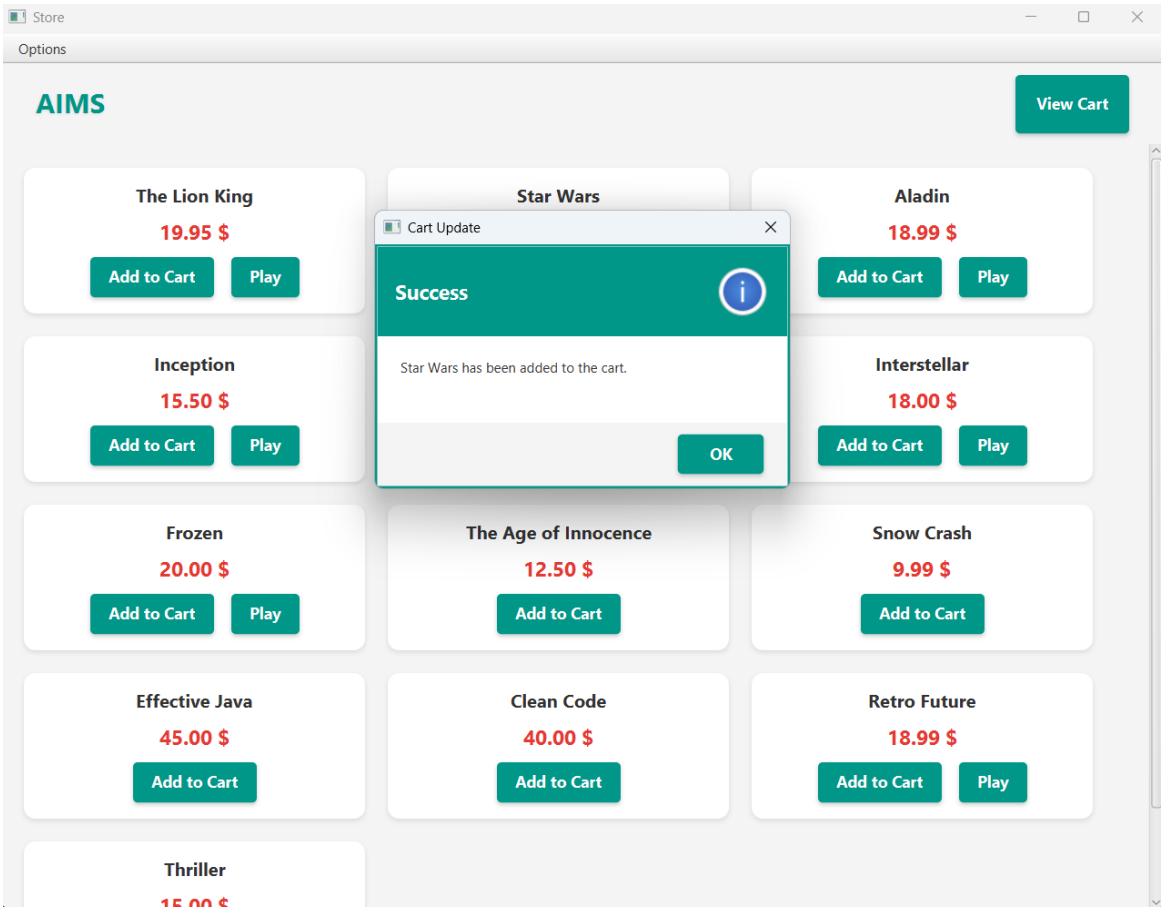

Figure 4: Successfully added media to cart

- When the user clicks the **Add to Cart** button, a dialog box titled "Cart Update" appears to confirm that the selected media has been successfully added to the cart.

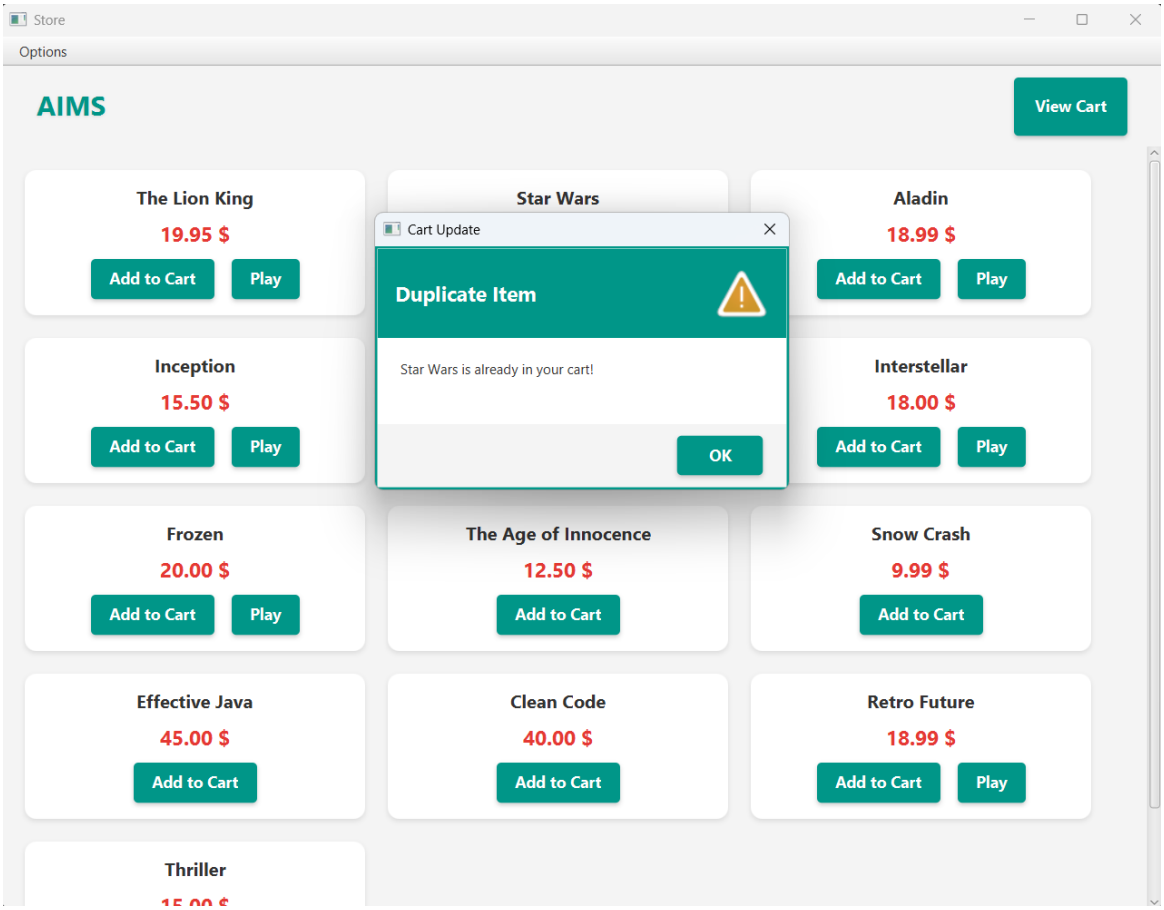

Figure 5: Duplicate Item Warning

- When the user selects an item to add, the system checks if the item is already present in the `itemsOrdered` list. If the item has already existed in the cart, the system will show a message.

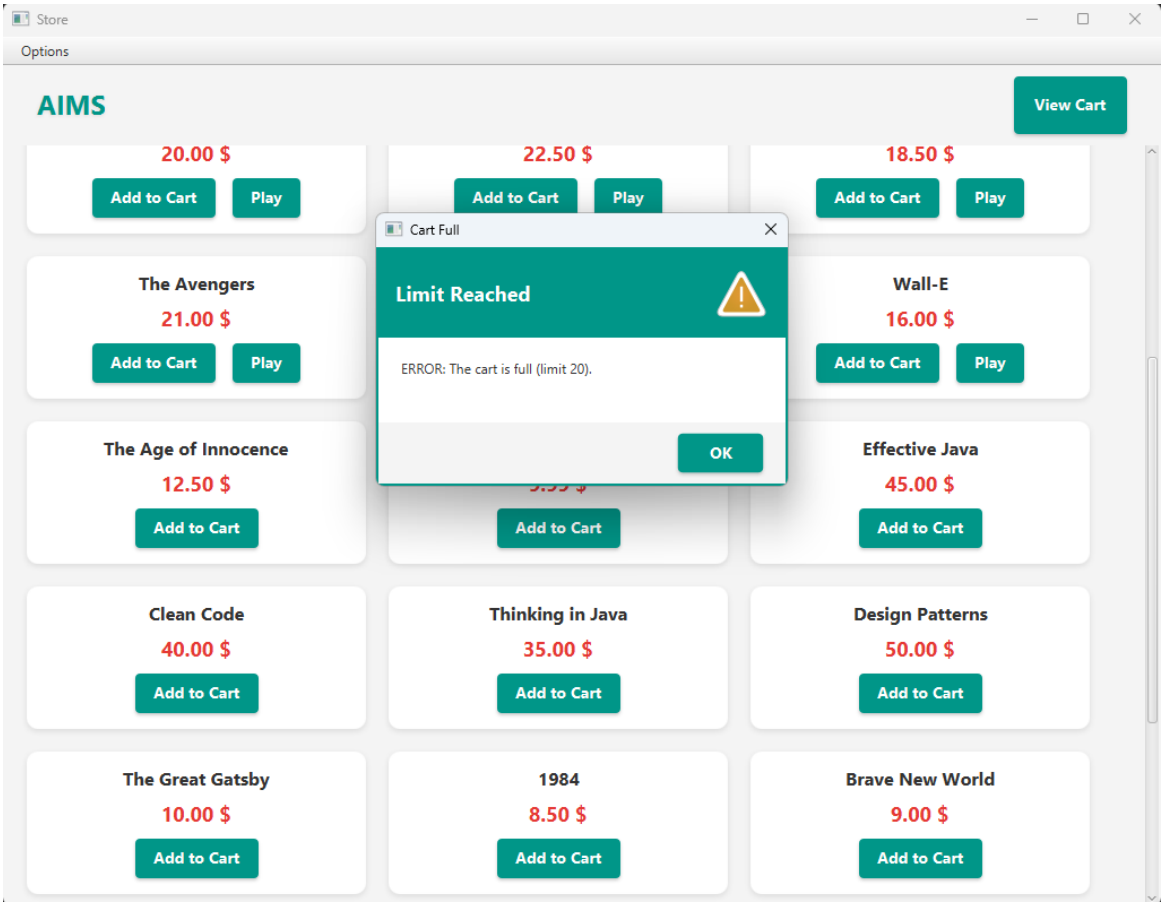

Figure 6: Cart Limit Warning

- If the number of media in the cart has reached 20, the system will show a warning.

3.3 "Play" Functionality

- **Constraint:** The system strictly enforces the `Playable` interface.

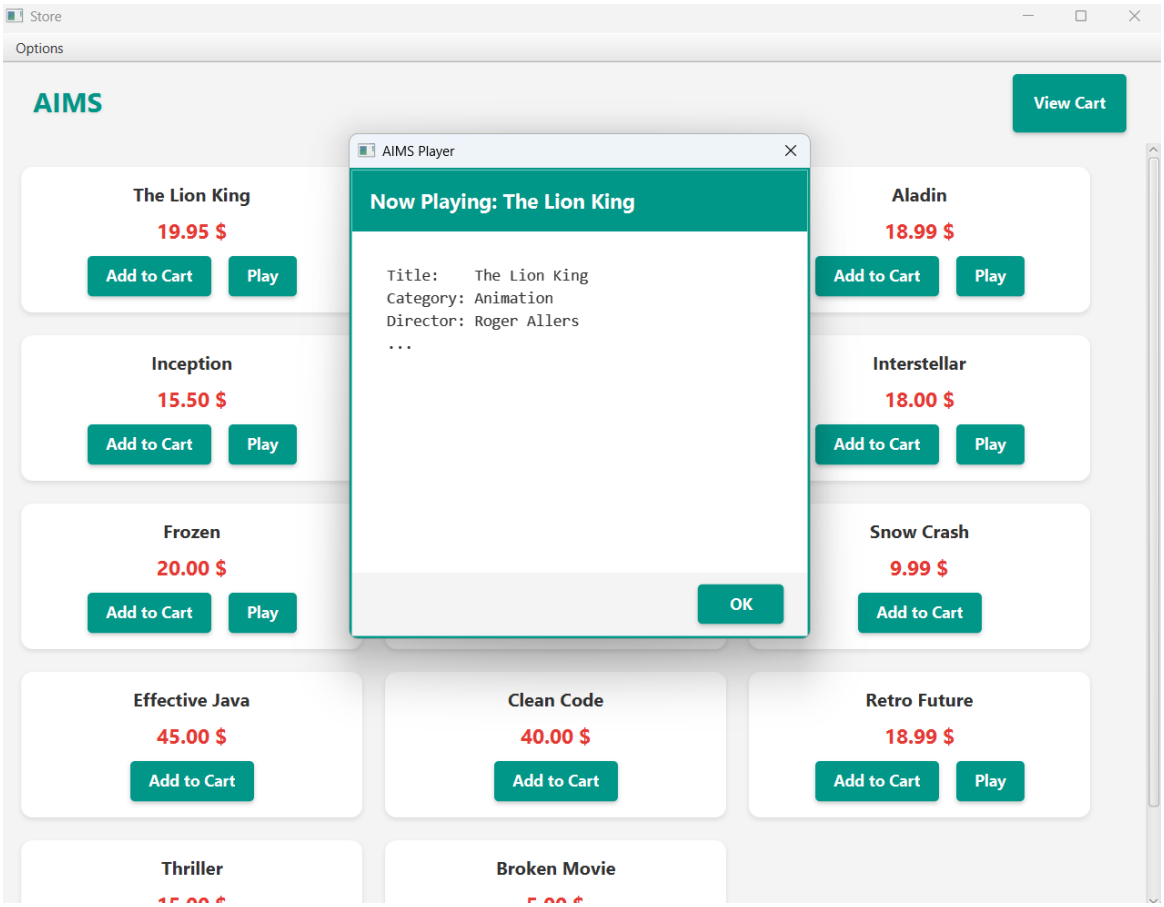

Figure 7: Popup dialog when playing media

- If a **CD or DVD** is selected, the "Play" button is enabled, and clicking it triggers a popup dialog that shows the info of the item.

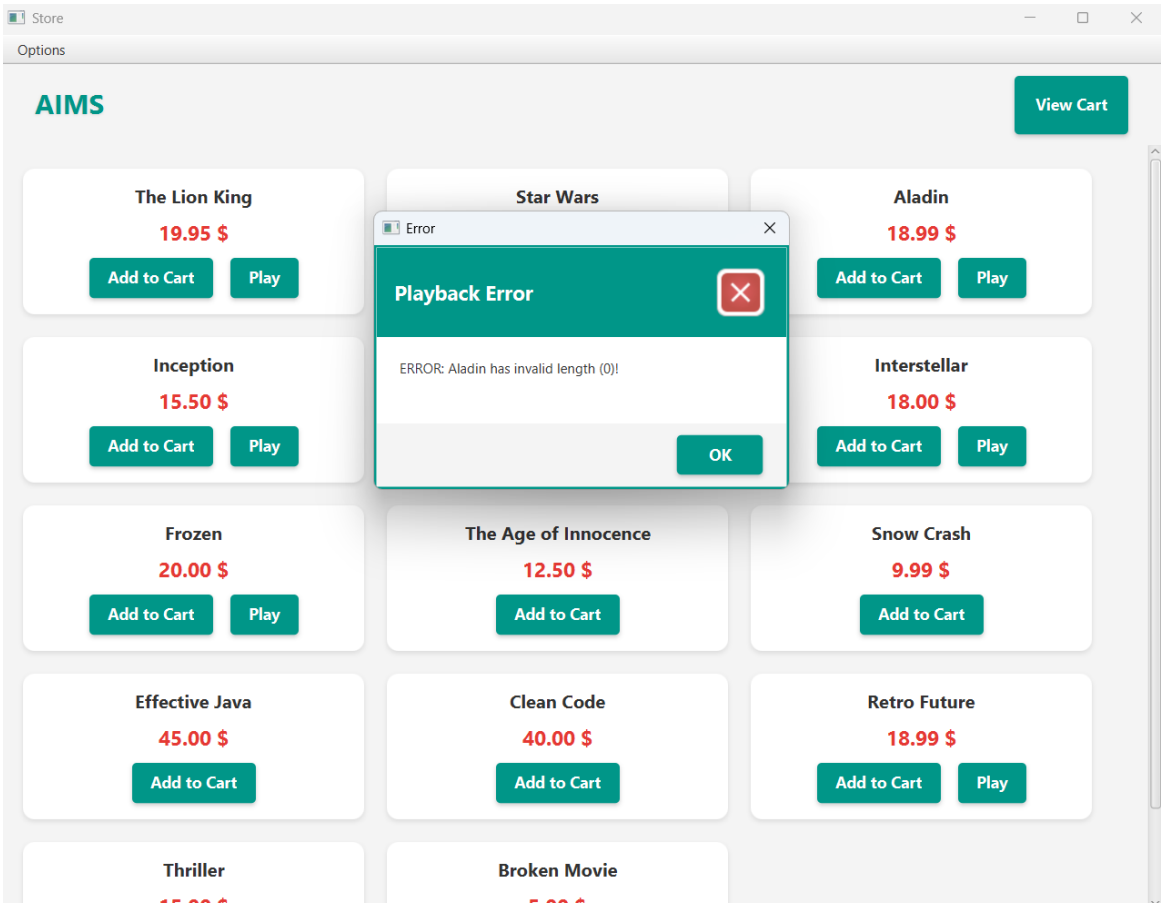

Figure 8: Playback Error Alert

- If a **CD or DVD** that has no length or negative length is selected, the AIMS Player will show a Playback Error.
- For the **Book**, we cannot play under any circumstances.

3.4 "View Cart" Functionality

This section analyzes the logic within `CartScreenController.java`.

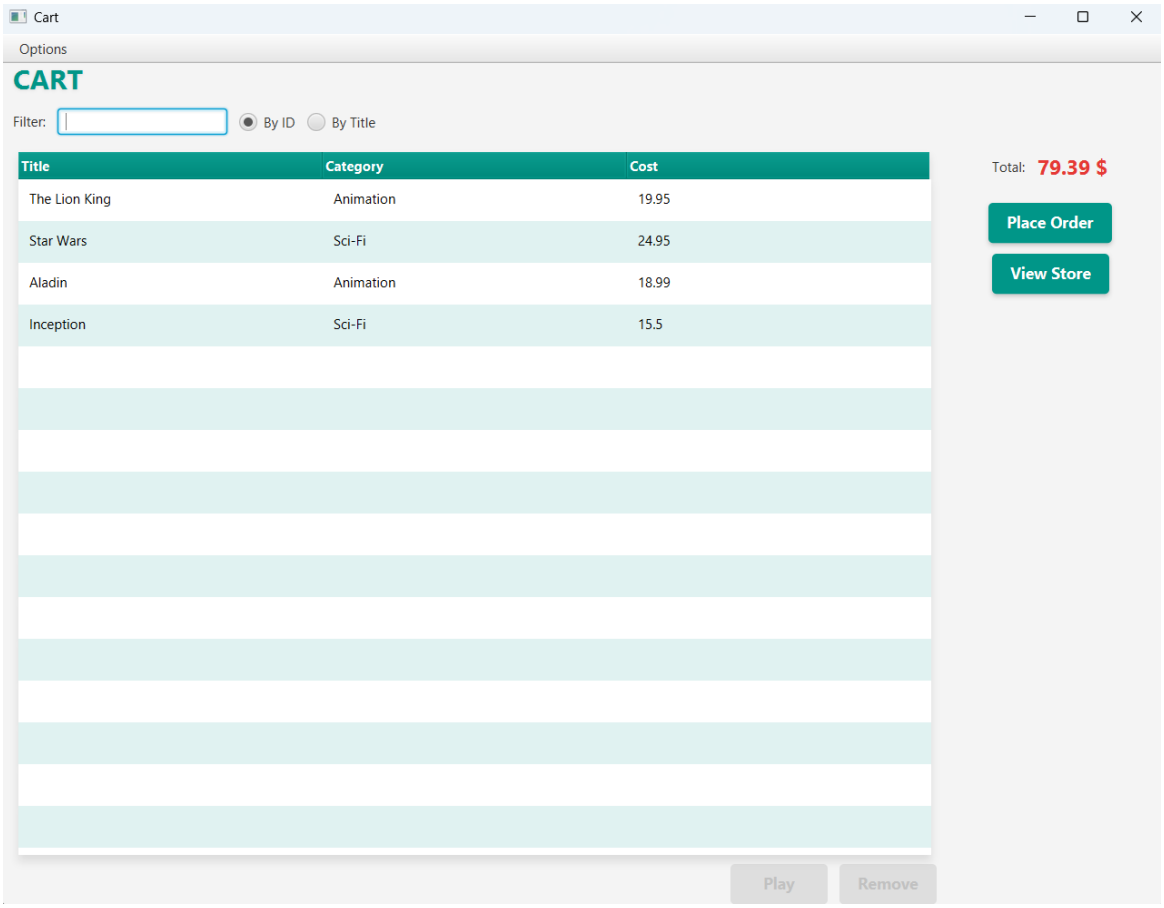

Figure 9: Cart View

- When we click **View Cart**, we can see the cart window and the list of media the user has chosen. There, users can choose filtering by ID or Title, see total cost, place order, or come back to the store.

3.4.1 Filtering (Search by ID and Title)

The system uses a `FilteredList` wrapper around the `ObservableList` of media. A listener is attached to the `tfFilter` text field. As the user types, the predicate updates in real-time, showing only rows where the ID or Title matches the keyword (case-insensitive).

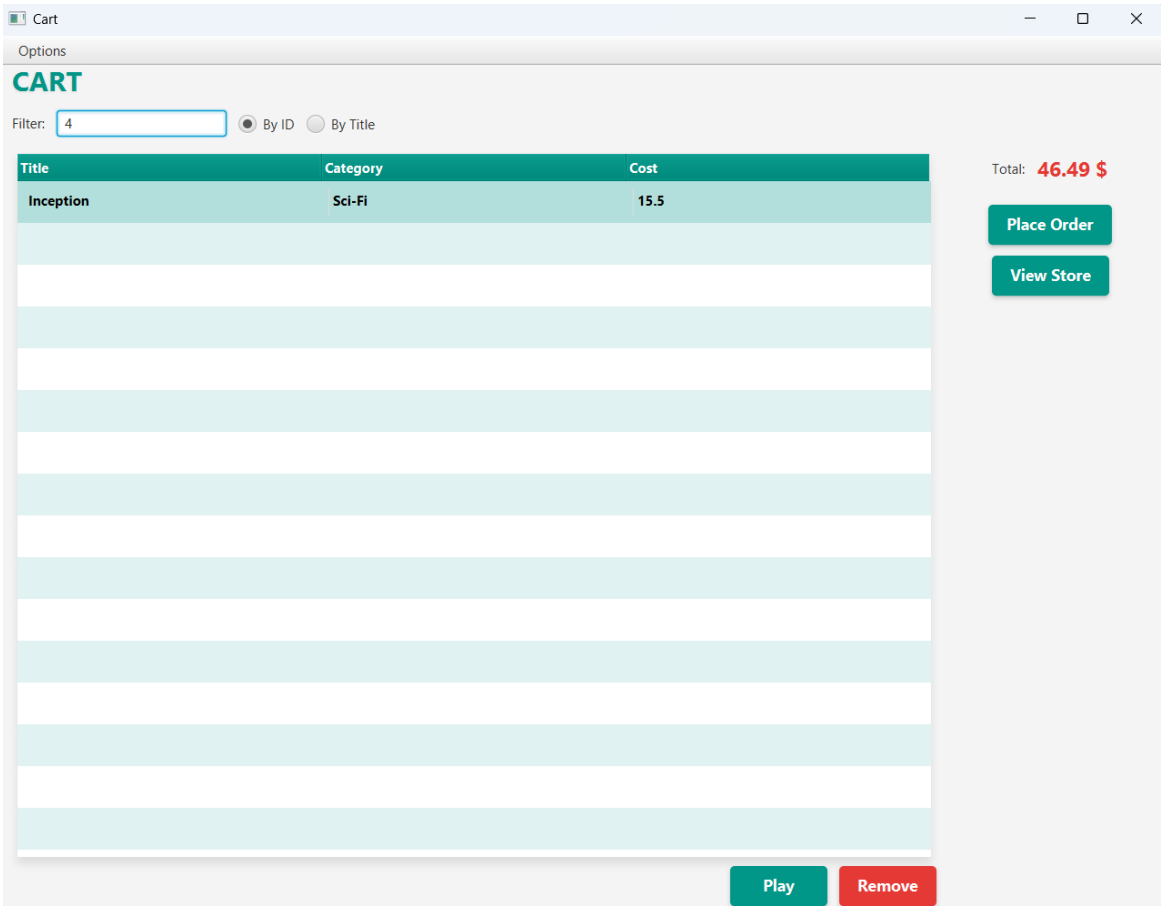

Figure 10: Filtering by ID

- When users choose to **by ID**, the AIMS returns the item corresponding to that ID.

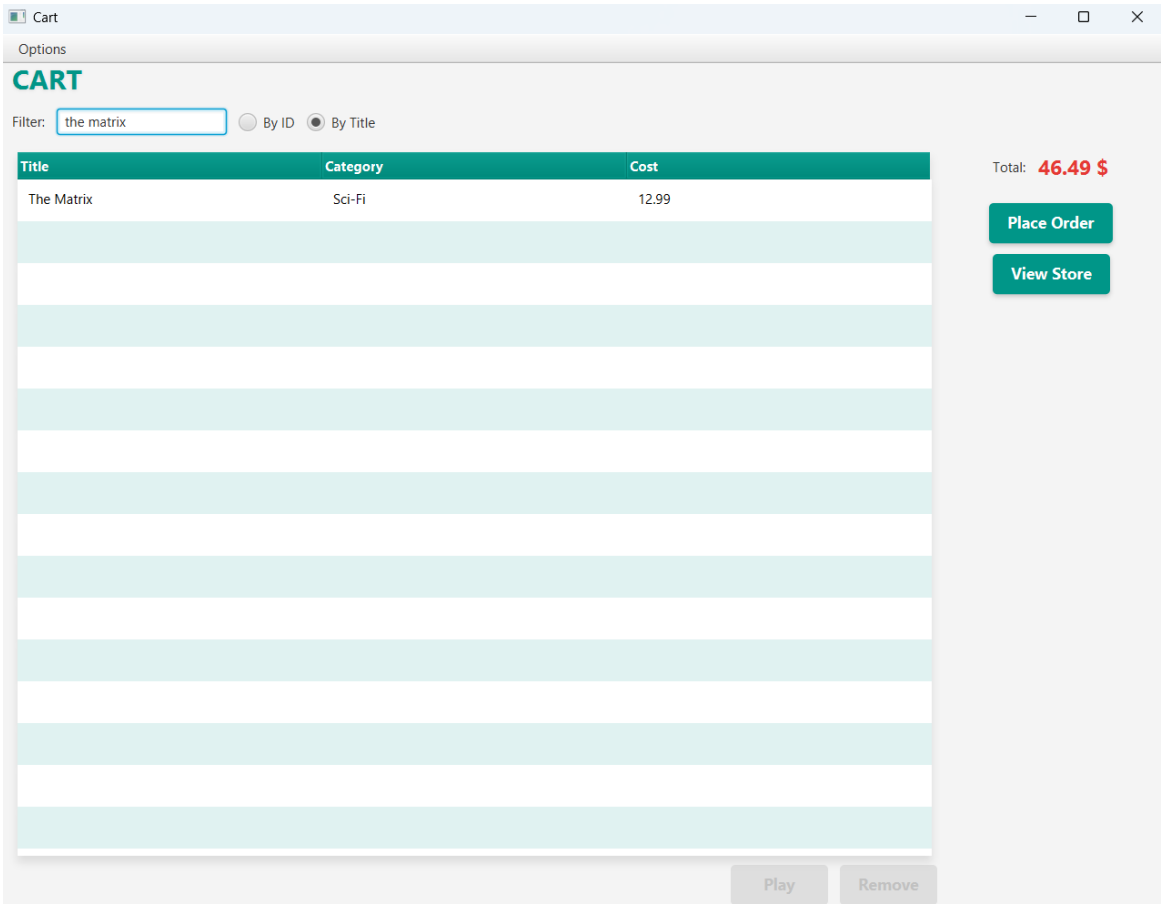

Figure 11: Filtering by Title

- When users choose to **by Title**, the AIMS returns the item corresponding to that title (case-insensitive).

3.4.2 Place Order

When the "Place Order" button is clicked:

1. The system calculates the total cost using `cart.totalCost()`.
2. An `Alert` (Information type) is displayed: "Thank you for your order!".

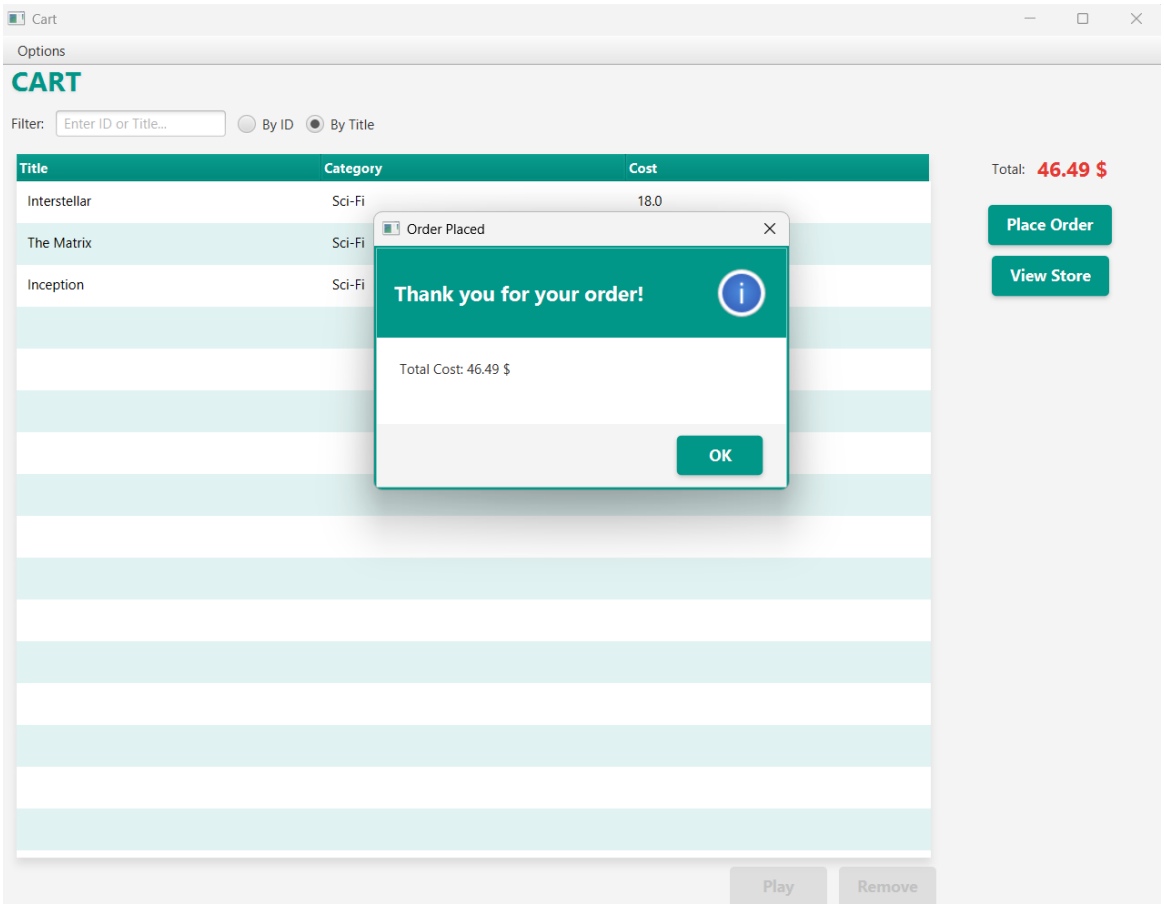

Figure 12: Order Confirmation

3. The cart is cleared using `cart.empty()`, resetting the view.

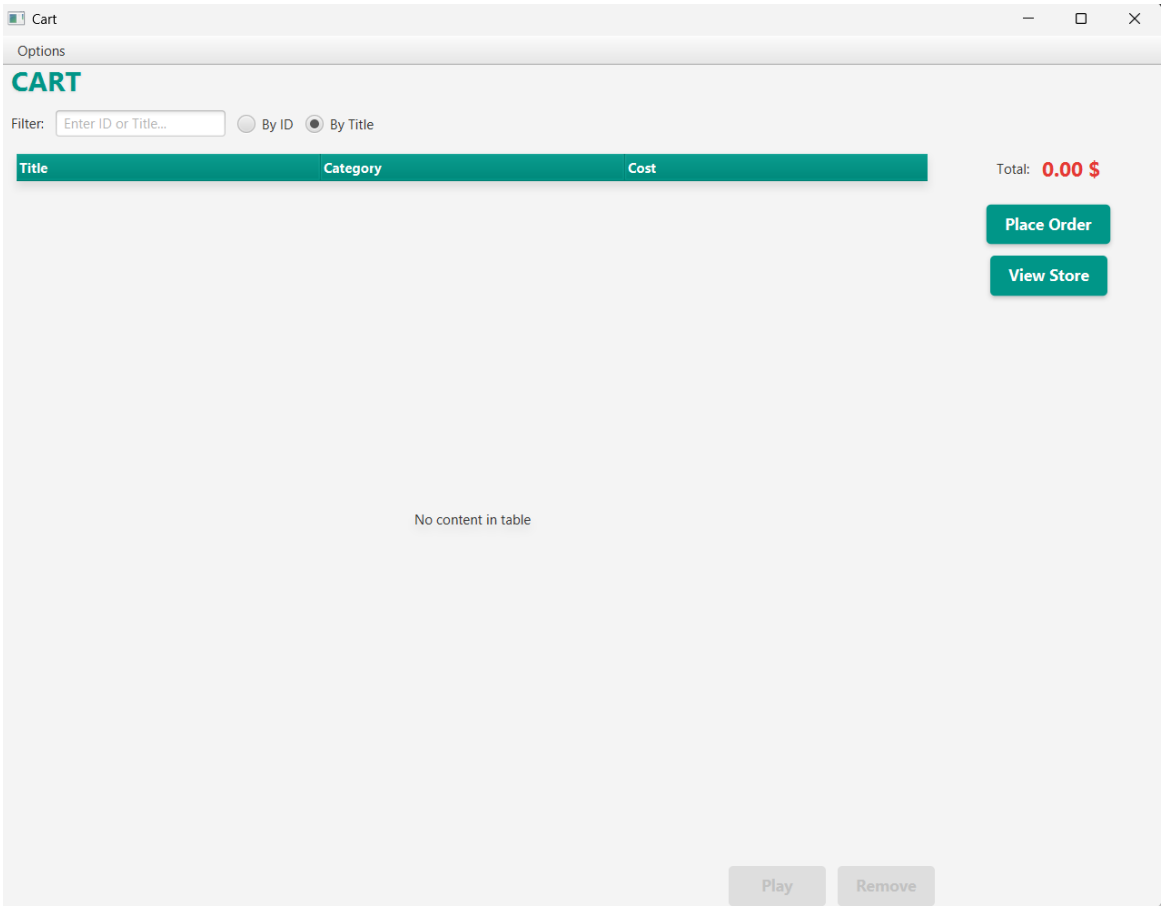

Figure 13: Cart Cleared

3.4.3 Navigation

The menu bar provides a "View Store" option which triggers `btnViewStorePressed`. This closes the current Cart window and opens a new `StoreScreenFX` stage, allowing users to add more items into the cart.

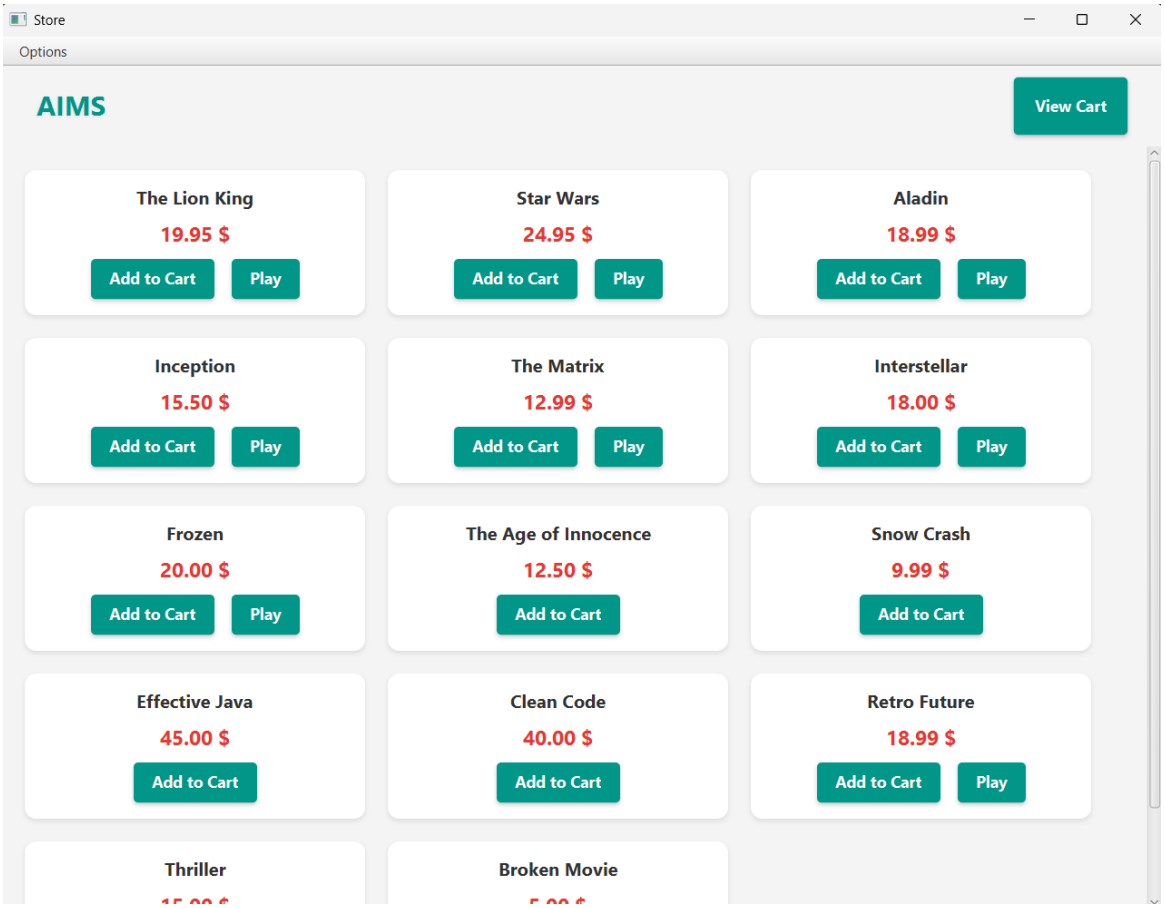

Figure 14: Returning to Store

- After that users can add more items to their cart.

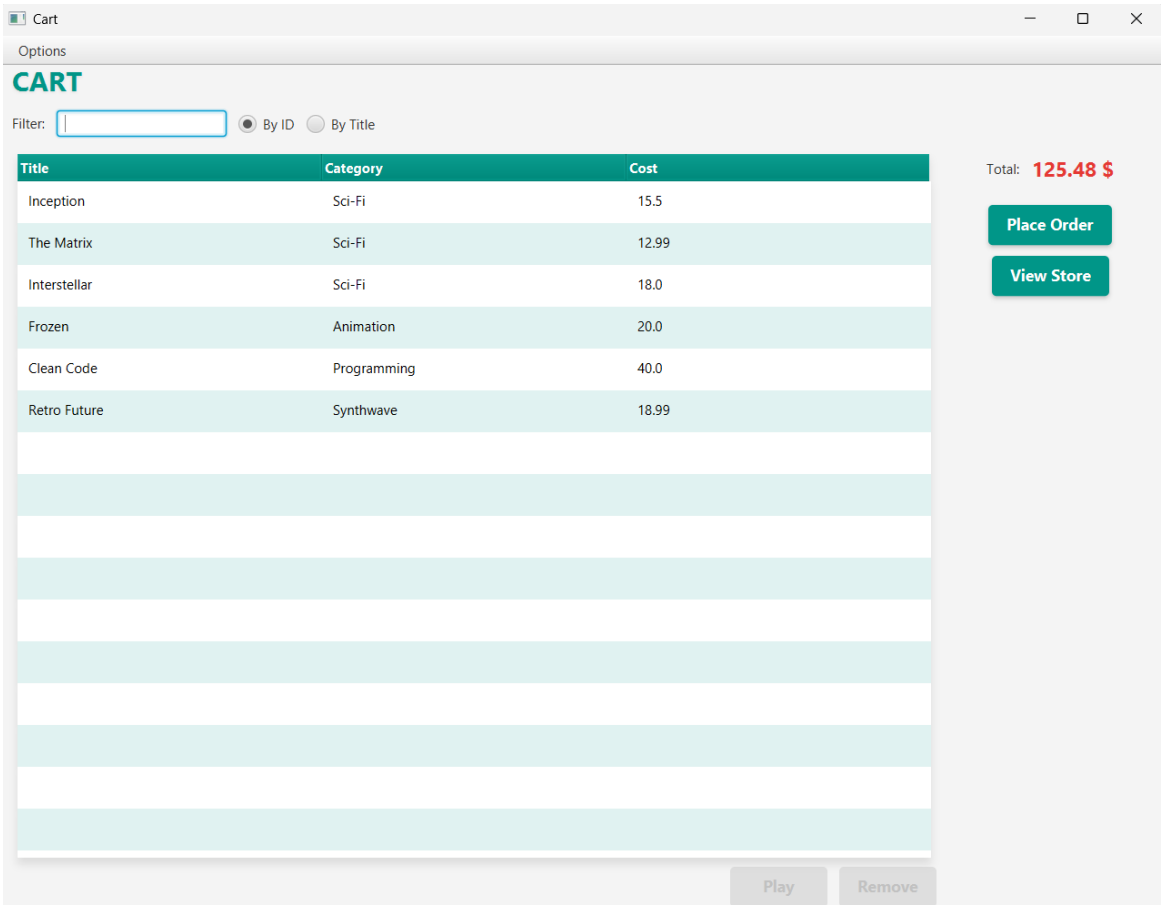

Figure 15: Continuing Shopping

3.4.4 Item Selection Handling

- **Book Selected:** The "Remove" button is enabled; "Play" is disabled.

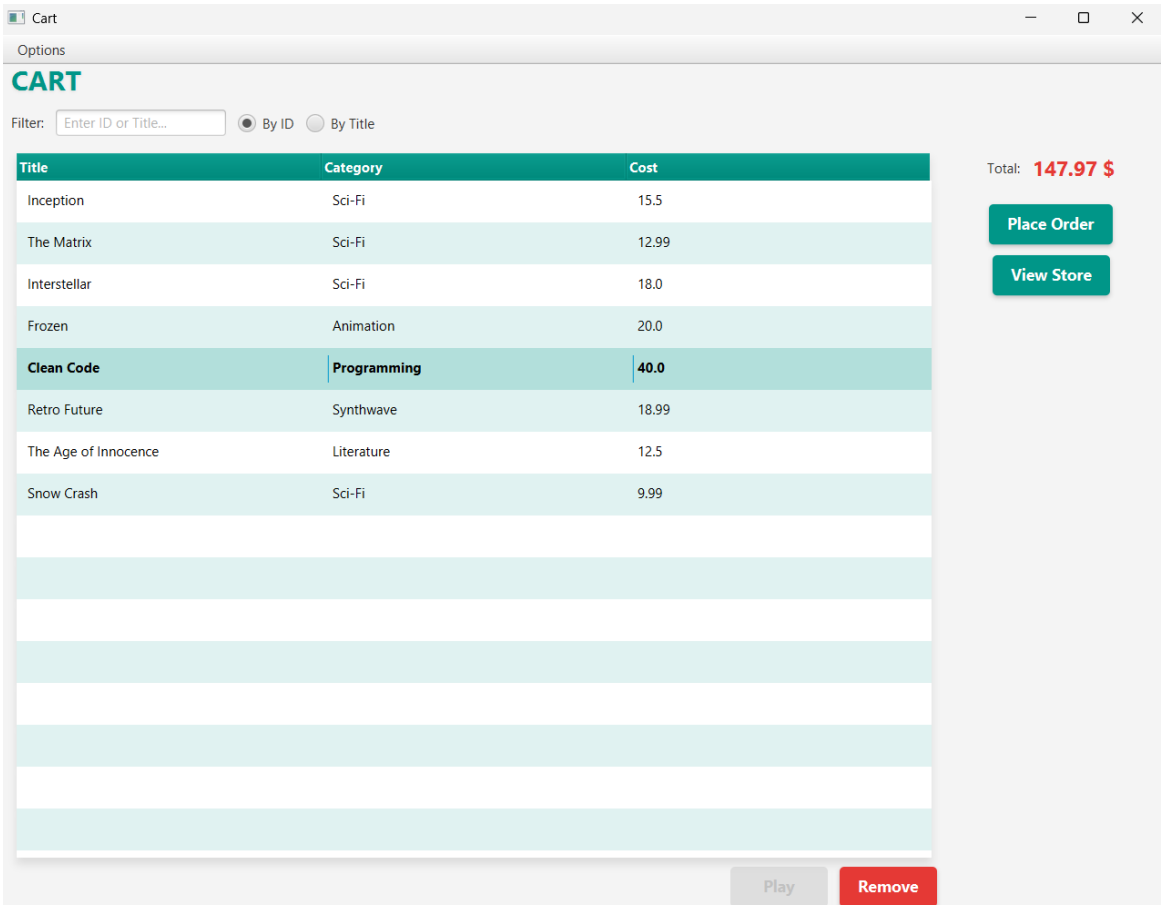

Figure 16: Selection Behavior: Book

- **DVD/CD Selected:** Both “Remove” and “Play” buttons are enabled.

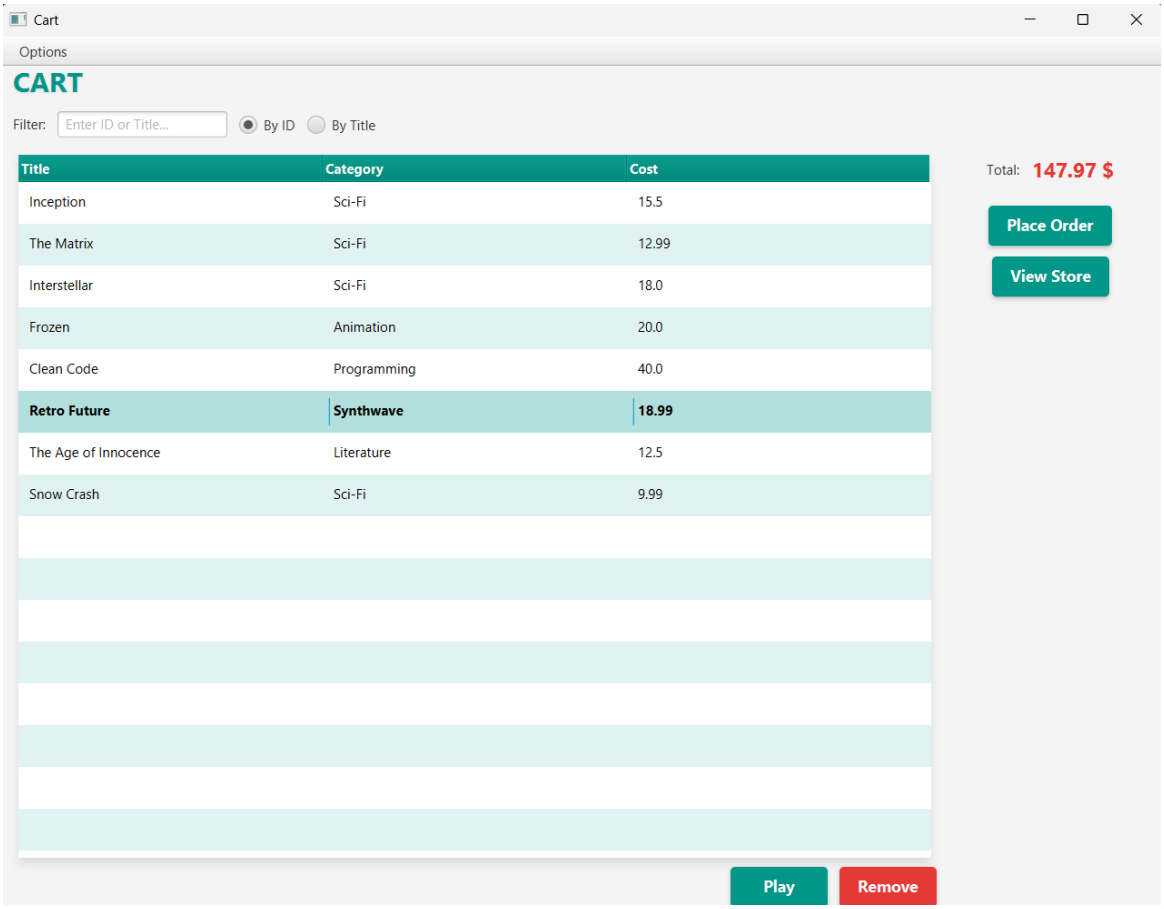

Figure 17: Selection Behavior: Playable Media

- **DVD/CD Selected:** When users click **Play** button, the system shows the information of the item.

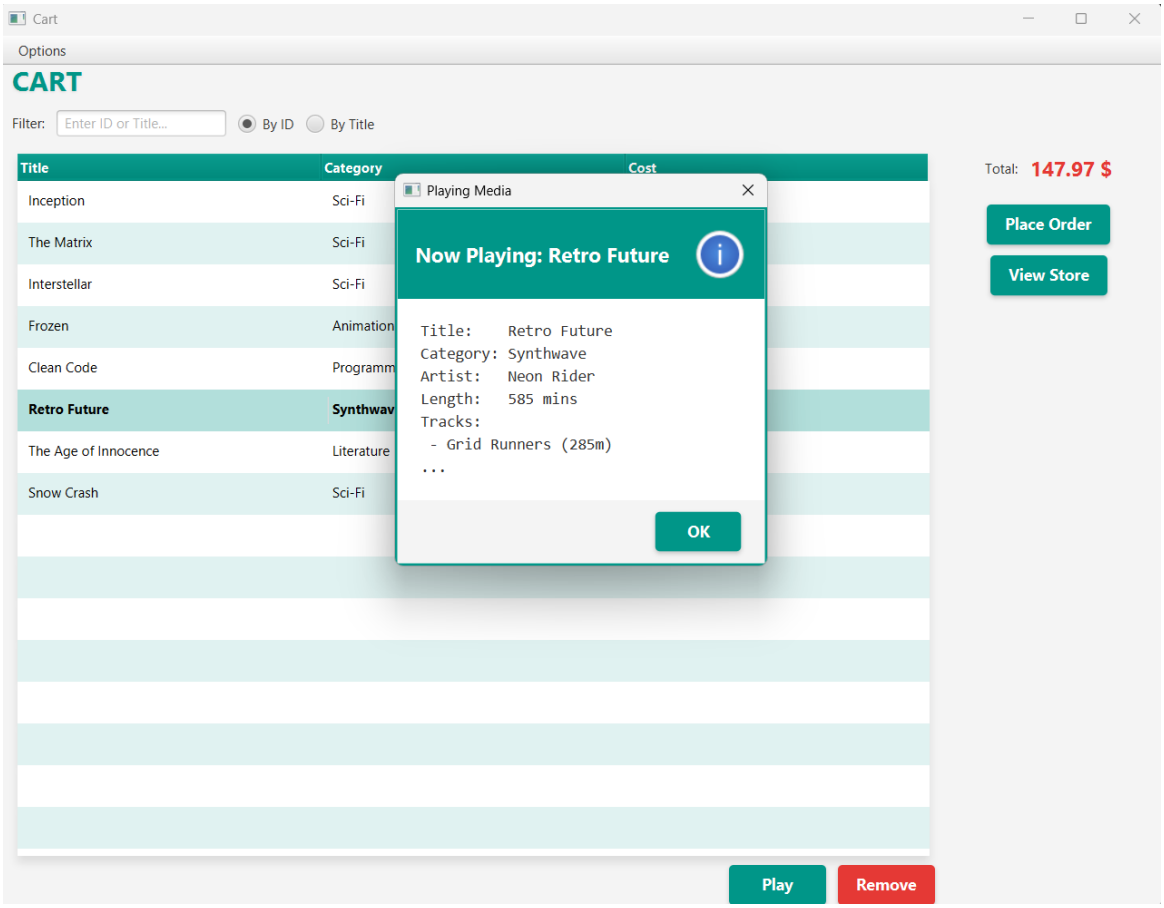

Figure 18: Playing from Cart

- When users click **Remove** button, the system simply erase the item from the cart.

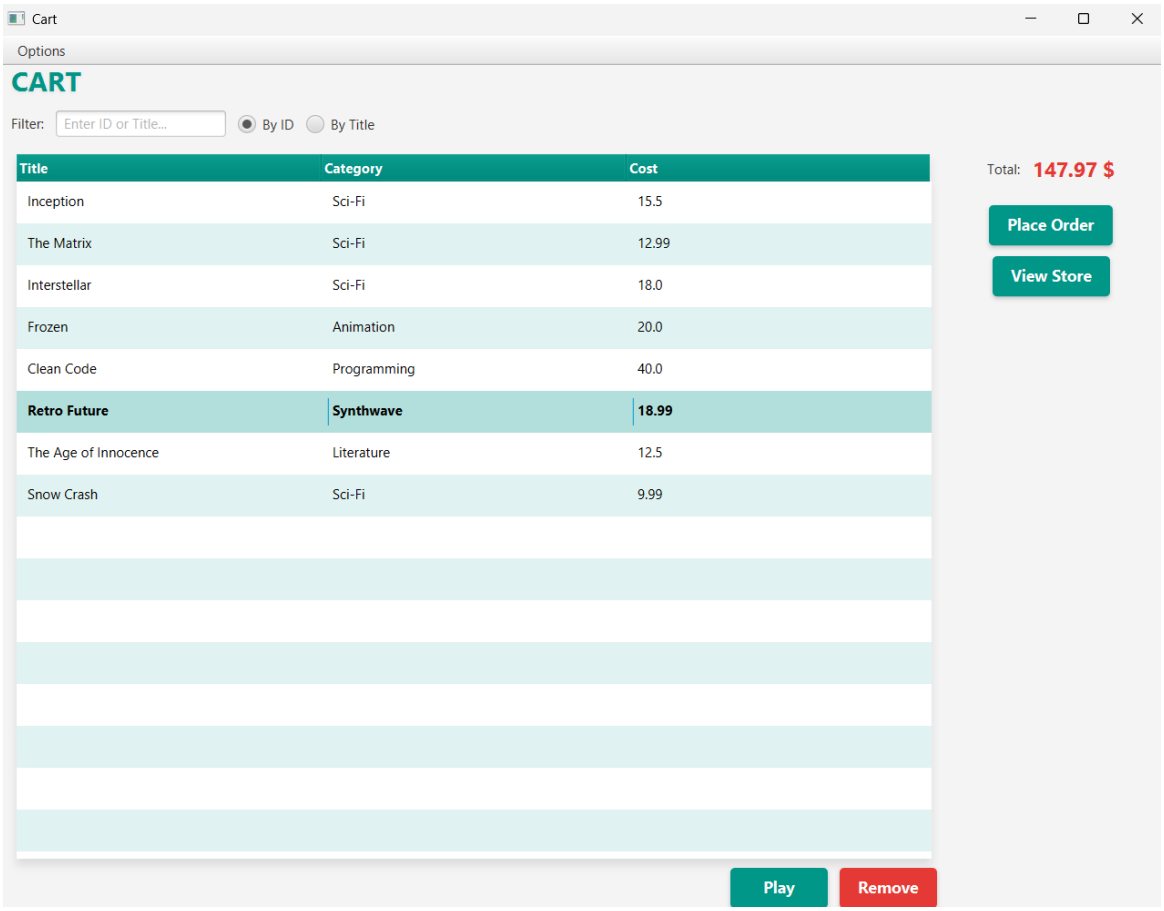

Figure 19: Cart Before Removal

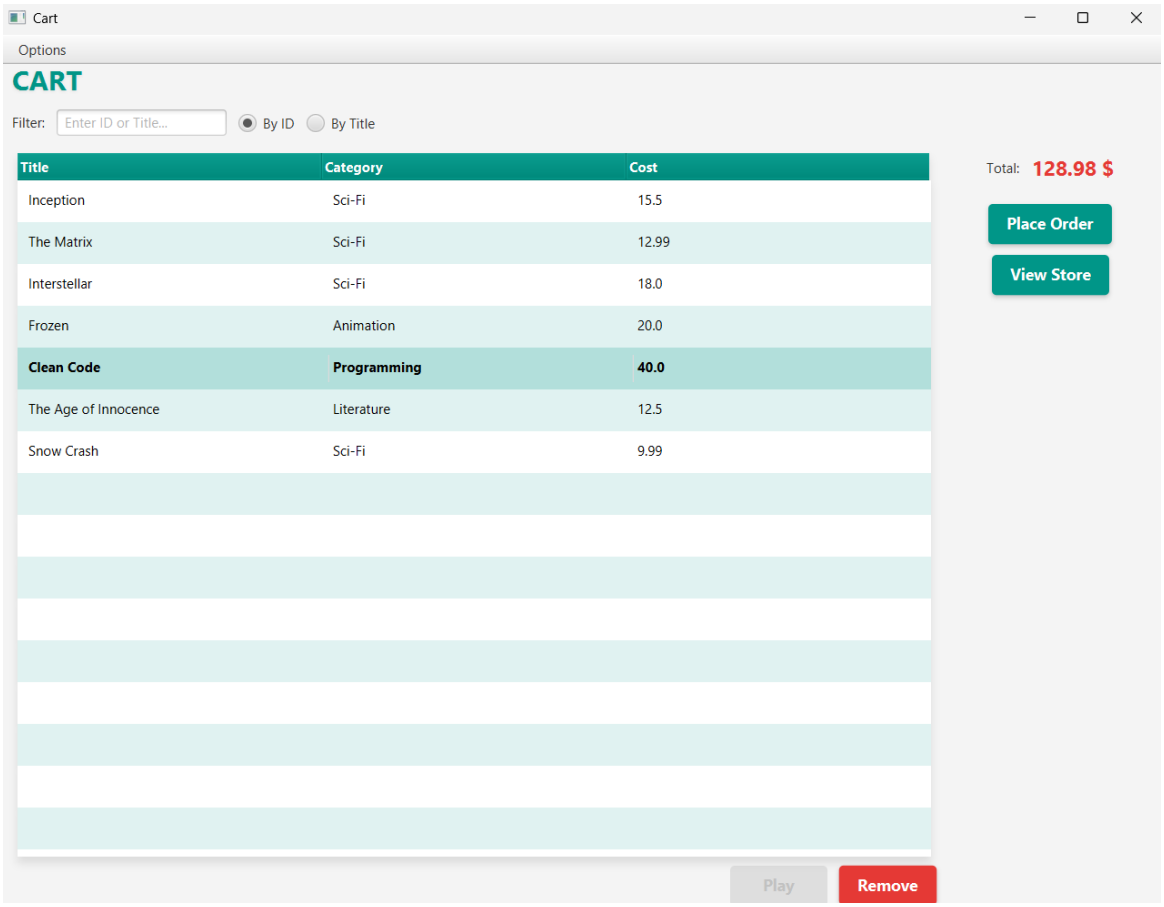

Figure 20: Cart After Removal

4 Exception Handling in AIMS

The AIMS system implements a comprehensive exception handling strategy to manage runtime errors, invalid user inputs, and business logic violations.

4.1 Purpose

Exception Handling helps the system to:

- Prevent abrupt termination (crashes) when errors occur during execution.
- Notify users of errors in a friendly and understandable manner.
- Ensure the stability and maintainability of the software.

4.2 Summary of Regular Exceptions

Exception / Error	**Type**	**Trigger Condition**	**Handling Strategy**
`PlayerException`	Custom	Media length is non-positive (≤ 0) or file is corrupted.	Catches exception and displays a **Red Error Alert**.
`NumberFormat...`	Standard	User inputs non-numeric text (e.g., "abc") into Cost/Length fields.	Catches exception and shows an Information Alert.
`LimitExceeded`	Logical	Cart reaches the maximum capacity (20 items).	Aborts action and displays a **Warning Alert**.
`DuplicateItem`	Logical	User adds an item already present in the cart.	Checks `.contains()` and displays a **Warning Alert**.

Table 1: Summary of Exception Handling Strategies in AIMS

4.3 Implementation Details

4.3.1 Handling Media Playback Errors

If errors happen when playing media (such as an invalid length), a dialog box displays the specific error message, ensuring the application handles the exception gracefully without crashing.

```
    try {
        ((Playable) media).play();

        showPlayDialog(media);

    } catch (PlayerException ex) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Error");
        alert.setHeaderText("Playback Error");
        alert.setContentText(ex.getMessage());
        styleAlert(alert);
        alert.showAndWait();
    }
});
```

Figure 21: Try-catch block handling PlayerException

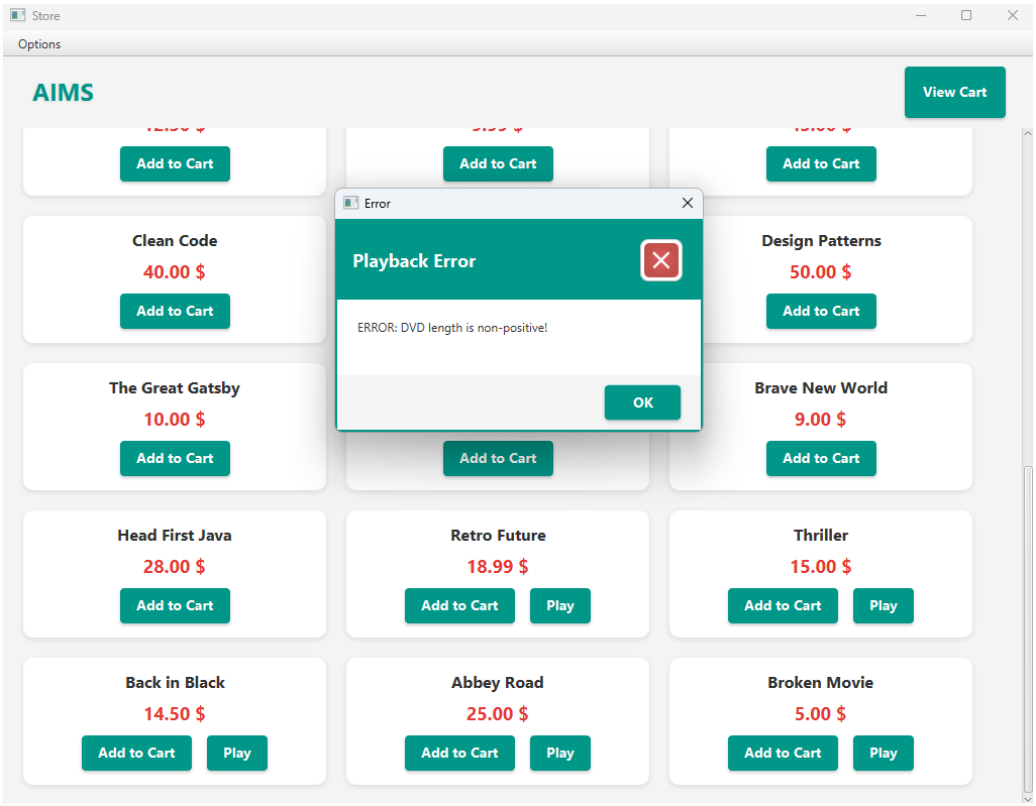

Figure 22: Alert displayed for Media with invalid length

4.3.2 Handling Cart Additions

Two distinct exceptions were implemented to segregate these error cases. Relying on a single `IllegalArgumentException` would have created ambiguity, preventing the user from understanding why the item could not be added.

```
    try {
        cart.addMedia(media);

        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Cart Update");
        alert.setHeaderText("Success");
        alert.setContentText(media.getTitle() + " has been added to the cart.");
        styleAlert(alert);
        alert.showAndWait();

    } catch (LimitExceededException ex) {
        Alert alert = new Alert(Alert.AlertType.WARNING);
        alert.setTitle("Cart Full");
        alert.setHeaderText("Limit Reached");
        alert.setContentText(ex.getMessage());
        styleAlert(alert);
        alert.showAndWait();

    } catch (DuplicateItemException ex) {
        Alert alert = new Alert(Alert.AlertType.WARNING);
        alert.setTitle("Cart Update");
        alert.setHeaderText("Duplicate Item");
        alert.setContentText(ex.getMessage());
        styleAlert(alert);
        alert.showAndWait();
    } catch (Exception ex) {
        ex.printStackTrace();
    }
});
```

Figure 23: Try-catch block handling LimitExceed and DuplicateItem Exception

4.3.3 Handling Input Format Errors

When a number is entered into the search filter, the function works normally. However, upon detecting non-numeric input, the system throws an exception and displays a notification message to the user.

```java
try {
    int id = Integer.parseInt(keyword);

    filteredItems.setPredicate( Media media -> media.getID() == id);

} catch (NumberFormatException e) {
    Alert alert = new Alert(AlertType.INFORMATION);
    alert.setTitle("Input Error");
    alert.setHeaderText("Invalid ID Format");
    alert.setContentText("Please enter a valid numeric ID.\nYou entered: " + keyword);
    setStyle(alert);
    alert.showAndWait();
    filteredItems.setPredicate(null);
}
```

Figure 24: Try-catch block handling NumberFormatException

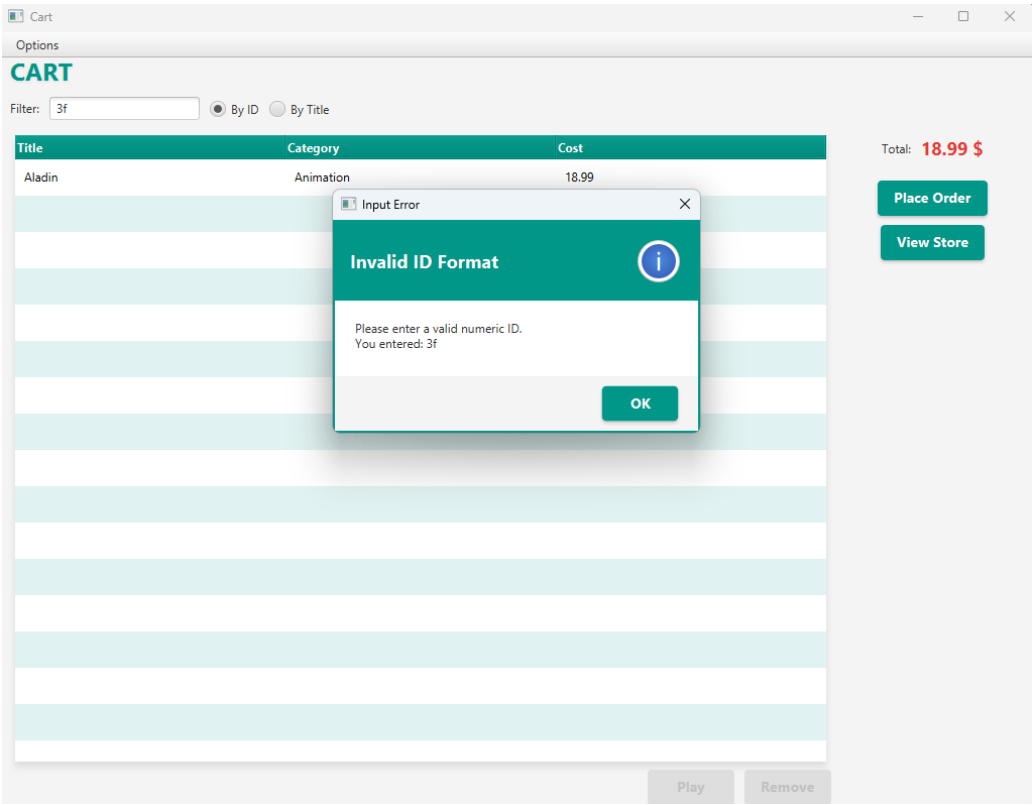

Figure 25: Alert displayed when entering a string instead of a number

4.4 Benefits of Exception Handling

- **User Friendliness:** Instead of crashing or showing cryptic errors, the user receives clear, actionable notifications.
- **Maintainability:** Error handling logic is isolated, allowing developers to modify it without affecting the main system logic.

- **Data Integrity:** Prevents invalid actions (e.g., adding invalid items) from corrupting the system state.

4.5 Conclusion

Exception Handling is a crucial component that ensures system stability, enhances the user experience, and facilitates easier software maintenance and development. The AIMS system successfully implements these principles alongside core OOP concepts to meet all Lab 05 requirements.